

CATALOGGUARD LITE

상품 카탈로그 데이터 품질 검수 서비스

프로젝트 한 줄 소개

서로 다른 합성 공급사 CSV를 JSON 프로파일로 표준화하고 품질 오류를 검수한 뒤, PostgreSQL staging 에서 Preview 승인 기반 Catalog Promotion 과 변경 Audit 까지 관리하는 Python·FastAPI 백엔드 MVP 입니다.

프로젝트 요약

항목	내용
형태	개인 프로젝트 · 취업 포트폴리오용 MVP 고도화 (2026.08 기준)
담당	검수 규칙, FastAPI API, PostgreSQL, Redis·Celery, ETL 프로파일 2 종, Promotion/Audit, Streamlit, 테스트·CI
핵심 기술	Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Redis, Celery, pandas, Streamlit, Playwright
운영·검증	Docker Compose, Dockerfile.aws, GitHub Actions, Railway, AWS EC2·RDS staging

검증된 현재 상태

1042 passed 최근 전체 pytest	Run 30737512739 test · browser-e2e 성공	Chromium E2E 2 개 Promotion 포함 브라우저 검증	AWS Runtime CI build · UID · migration · /health
------------------------------------	---	---	--

핵심 성과

검수 결과 저장·검색·상세 조회와 SHA-256·inspection_version·DB 제약을 결합해 동일 CSV 재검수와 동시 요청 중복을 방어하였습니다.

Redis·Celery 비동기 검수, 공급사 Profile 기반 ETL, PostgreSQL staging 적재를 구현하고 정상·reject·summary 산출물과 트랜잭션 무결성을 검증하였습니다.

Promotion Preview → 사용자 승인 → expected hash 재검증 → 운영 상품 반영 → Promotion Run·Audit·History 조회까지 안전한 반영 흐름을 연결하였습니다.

Playwright Chromium E2E 와 AWS Docker runtime smoke 를 GitHub Actions 에 추가해 실제 사용자 흐름과 build 후 runtime 오류를 자동 회귀 검증합니다.

GitHub [psy0635-ctrl/catalogguard-lite](#) Demo [CatalogGuard Lite Streamlit 앱](#)

1. 문제 정의와 사용자 가치

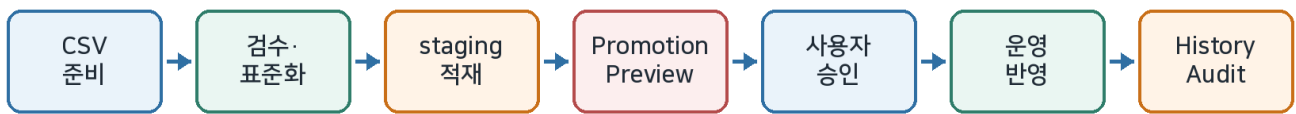
기능 수를 늘리기보다 상품 데이터가 왜 검수·표준화되어야 하고 운영 반영 전에 무엇을 확인해야 하는지부터 정의했습니다.

문제 정의

쇼핑몰 상품 카탈로그는 공급사와 운영 담당자마다 컬럼명·표기 방식이 달라 같은 의미가 여러 형태로 저장될 수 있습니다. 오류가 누적되면 검색·필터·재고 관리·상품 노출의 정확도가 낮아지고, 검수되지 않은 staging 데이터가 바로 운영 상품에 반영되면 변경 원인을 추적하기도 어렵습니다.

반복되는 문제	예시	서비스의 대응
형식 불일치	price 에 문자, 필수값 누락	업로드 검증과 규칙 기반 오류 탐지
표기 불일치	black·블랙·BLACK	색상·사이즈 별칭을 표준값으로 정규화
관계 오류	같은 그룹에 다른 카테고리	product_group_id 단위로 그룹 전체 비교
공급사 구조 차이	vendor_sku / style_id	JSON 프로필로 표준 컬럼에 매핑
적재 추적 부족	표준 CSV 가 파일로만 남음	PostgreSQL staging 배치·상품 이력 저장
운영 반영 위험	검수 후 데이터가 변경됨	Preview Hash 재검증·승인·Audit 으로 반영 보호

사용자 흐름



사용자가 받는 결과

검수 상태·오류 이유·수정 권장사항과 ETL 배치·상품, Promotion Preview 의 신규/수정/변경 없음 건수, 반영 후 실행 이력과 상품별 Audit 을 Streamlit 에서 확인할 수 있습니다.

MVP 범위

구현 완료 — 규칙 기반 검수, 검수 이력, 비동기 검수, Profile ETL, staging 적재·조회, Promotion Preview·승인·Audit·History, Streamlit UI, 브라우저 E2E.

의도적 제외 — 인증·권한·rate limit, 외부 공급사 실연동, streaming·중분 ETL, 운영 Redis·Celery 배포 자동화는 현재 MVP 범위에 포함하지 않았습니다.

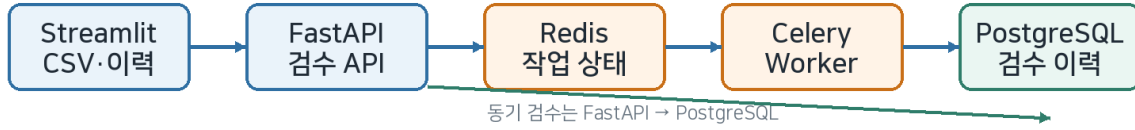
2. 시스템 구조와 책임 분리

화면·API·Service/Query·작업 큐·DB 의 역할을 분리하고 Streamlit 에서 DB 를 직접 수정하지 않도록 구성했습니다.

CatalogGuard Lite 전체 구조

검수 흐름과 ETL·Promotion 흐름을 분리하고 PostgreSQL을 공통 저장 기준으로 사용

동기·비동기 검수



공급사 ETL → staging → 안전한 Promotion



UI는 DB에 직접 쓰지 않고 CatalogGuardApiClient → FastAPI → Service/Query 계층을 거칩니다.

계층별 역할

구성	주요 책임	설계 판단
Streamlit	검수 결과·이력, ETL/Promotion History·Audit UI	DB 직접 접근 없이 CatalogGuardApiClient 만 사용
FastAPI	동기·비동기 검수, ETL 조회, Promotion API	요청 검증·상태 코드·Pydantic 응답 계약
Service / Query	검수·Promotion 트랜잭션 / 읽기 전용 조회	쓰기와 조회 책임 및 transaction boundary 분리
Redis·Celery	job 상태와 백그라운드 검수	긴 검수를 HTTP 요청 수명과 분리
PostgreSQL	검수·ETL staging·운영 상품·Run·Audit	constraint·unique·FK·transaction 으로 무결성 보강
GitHub Actions	DB/Redis/E2E/브라우저/AWS runtime 자동 검증	과거 결함을 재현 가능한 회귀 검증으로 전환

핵심 기술 판단

단일 검수 기준 — 검수 규칙은 FastAPI 서비스에서 한 번 실행하고 Streamlit 은 저장된 결과를 표시하여 이중 검수를 제거하였습니다.

읽기/쓰기 분리 — History Query Service 는 commit·rollback 없이 pagination·count 를 담당하고 Promotion Service 만 운영 데이터 변경 transaction 을 가집니다.

상태 분리 — 검색·배치 선택·Preview·승인·History cache 상태를 분리하고 성공 시 필요한 cache 만 무효화하여 stale UI 를 방지하였습니다.

3. 핵심 검수 기능과 결과 계약

상품 한 행의 오류뿐 아니라 패션 옵션·그룹 관계와 개인정보 의심까지 규칙 기반으로 검수하고 같은 결과 계약으로 API 와 UI 에 전달합니다.

검수 범위

분류	검수 내용	대표 예시
입력·형식	필수값, category·stock·price·sale_price 형식	price="가격문의" → 형식 오류
중복	상품 ID, 상품명 후보, 완전 중복	동일 속성 상품 또는 ID 중복
패션 옵션	색상·사이즈 비표준, 옵션 조합 중복	블랙 → BLACK, medium → M
그룹 관계	같은 product_group_id 의 카테고리 불일치	TOP 과 BOTTOM 이 같은 그룹
가격	0 이하, 정상가·할인가 관계, 카테고리 이상치	price 50,000 / sale_price 60,000
내용·보안	상품명-카테고리, 금지어, 개인정보 의심	설명에 이메일·전화번호 포함

패션 데이터 정규화 예시

BLACK black / 블랙 / BLACK	M medium / M	XXL 2XL / XXL	FREE 프리사이즈 / FREE
------------------------------------	------------------------	-------------------------	-----------------------------

그룹 단위 판단과 결과 계약

ValidationIssue

모든 규칙은 severity, product_id, product_group_id, message 를 가진 공통 결과로 통일합니다. 그룹 카테고리 불일치는 다수결로 임의의 정답을 만들지 않고 불일치 사실을 참여 상품 전체에 연결하며 원본 행 순서를 유지합니다.

안전한 결과 표시

내부 상세 값은 JSON 구조로 저장해 심표·작은따옴표가 포함된 값도 안전하게 전달합니다.

손상되거나 예상하지 못한 JSON 은 화면 전체를 중단시키지 않고 기본 안내 문구로 처리합니다.

다운로드 CSV 의 =, +, -, @ 시작 값은 복사본에서만 안전하게 처리해 Excel 수식 삽입 위험을 줄였습니다.

개인정보 의심 결과는 UI/ETL reject 흐름에서 마스킹하여 테스트 데이터에 실제 개인정보를 사용하지 않습니다.

4. 동일 CSV 재사용·검수 버전과 비동기 처리

중복 저장 방지와 불필요한 재검수 방지를 분리하고, 긴 검수는 Redis·Celery 작업으로 HTTP 요청 수명에서 분리했습니다.

같은 파일을 안전하게 재사용하는 방법

문제	해결 방법	효과
같은 파일 반복 업로드	CSV bytes SHA-256	파일명과 무관하게 동일 내용 식별
규칙 변경 후 과거 결과 재사용	file_sha256 + inspection_version	버전이 바뀌면 같은 CSV 도 재검수
동시에 같은 요청	PostgreSQL partial unique index	사전 조회 이후 경쟁 요청도 DB 에서 차단
같은 세션 반복 제출	saved hash + 활성 job_id	불필요한 POST·GET 호출 감소

inspection_version 5

sale_price 형식·0 이하·정상가 초과 관계 규칙이 추가되었을 때 과거 결과가 잘못 재사용되지 않도록 검수 버전을 올렸습니다. 검수 버전은 기능 소 개용 숫자가 아니라 결과 재사용 정확성을 보장하는 식별자입니다.

Redis·Celery 비동기 검수

항목	구현 내용	검증
작업 상태	queued → running → succeeded / failed	Redis DB 1 상태 조회
작업 전달	FastAPI → Redis broker → Celery Worker	worker ping 및 실제 inspection task
결과 저장	완료 후 PostgreSQL inspection_runs/results	비동기 E2E 에서 저장 결과 확인
실패·재제출	failed 상태 후 사용자 명시적 재제출	중복 POST 방지·재시도 UI 테스트
임시 파일	성공·실패 후 공유 CSV 정리	E2E 종료 후 잔여 파일 없음

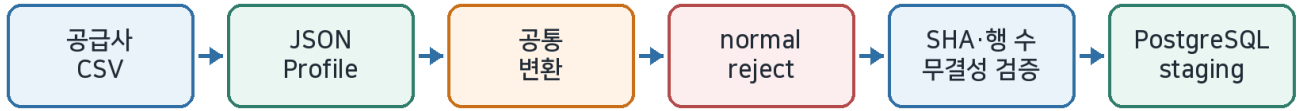
문제 해결 포인트

기존 방식 — 중복 여부를 저장 직전에 알았기 때문에 같은 CSV 도 전체 검수를 다시 수행했습니다.

개선 후 — 파일 검증·파싱 → hash/version 사전 조회 → 기존 결과 반환 또는 신규 검수 순서로 변경하고, 최종 중복은 DB unique constraint 로 방어했습니다.

5. 공급사 Profile ETL 과 PostgreSQL staging

공급사마다 다른 컬럼 구조를 JSON 프로필에 분리하고 공통 변환 코드로 정상·reject·summary 산출물과 staging 배치를 생성합니다.



합성 공급사 프로필 비교

프로필	원본 → 표준 컬럼	설계 의미
패션 공급사 v1	vendor_sku → product_group_id, product_id	하나의 키를 그룹/개별 ID에 사용
패션 공급사 v1	discount_price → sale_price	선택 할인 가격을 공통 필드로 변환
마켓플레이스 v1	style_id → product_group_id	상품 그룹 키를 별도로 유지
마켓플레이스 v1	sku_code → product_id	같은 그룹 아래 개별 SKU 유지
마켓플레이스 v1	promo_price → sale_price	할인 가격 관계 검수로 전달

staging DB 안전장치

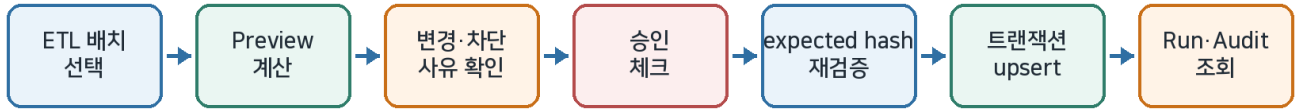
구성	역할	안전장치
etl_load_runs	파일·프로필·hash·적재 행 수를 배치 단위 저장	input hash + profile + version unique
catalog_products_staging	정상 상품을 batch ID와 연결	stock·price·sale_price CHECK
1:N 관계	배치 1 개에 상품 여러 개 연결	Foreign Key + DB cascade
트랜잭션	배치 생성·상품 저장을 하나로 처리	일부 실패 시 전체 rollback

무결성 검증

표준 CSV SHA-256 과 summary JSON 의 output_file_sha256, 실제 행 수와 loaded_rows 가 일치할 때만 적재합니다. 공급사 2 번째 프로필을 추가할 때 공통 profile_loader.py·transformer.py·pipeline.py 는 수정하지 않았습니다.

6. 안전한 Catalog Promotion 과 Audit

staging 데이터를 바로 운영 상품에 반영하지 않고 Preview·명시적 승인·Hash 재검증·트랜잭션·Audit 을 거치도록 설계했습니다.



Promotion 안전장치

위험	구현	사용자/DB 결과
무엇이 바뀌는지 모름	Preview 에서 insert/update/no-change 계산	상품별 변경 전·후와 반영 불가 사유 표시
Preview 후 데이터 변경	expected_preview_hash 재계산	불일치 시 409 preview_stale
실수로 즉시 반영	confirmed + 승인 checkbox	사용자 명시적 승인 전 실행 버튼 비활성
중간 실패로 일부 반영	PostgreSQL transaction	실패 시 전체 rollback
변경 원인 추적 불가	Promotion Run + 상품 Audit	이전/새 값과 관련 실행을 History 에서 조회

주요 API

엔드포인트	역할
POST /api/v1/etl-loads/{run_id}/promotion-preview	반영 예정 신규·수정·변경 없음과 Preview Hash 계산
POST /api/v1/etl-loads/{run_id}/promotions	confirmed + expected_preview_hash 로 실제 반영
GET /api/v1/catalog-promotions	Promotion 실행 이력 목록·상태 필터·페이지네이션
GET /api/v1/catalog-promotions/{id}	실행 상세
GET /api/v1/catalog-promotions/{id}/audits	상품별 변경 Audit 조회

용어 주의

이 프로젝트에서 Promotion 은 할인 행사가 아니라 ETL staging 의 검수된 상품을 운영 카탈로그에 반영하는 과정입니다.

7. ETL·Promotion 이력 UI 와 Chromium E2E

읽기 전용 조회 API 와 Streamlit 화면을 연결하고 실제 Chromium 에서 ETL → Preview → 승인 → Promotion → History → Audit 흐름을 검증했습니다.

조회·상태 관리

기능	구현 판단	검증
ETL 목록·검색	파일명·프로필 AND 검색, SQL LIMIT/OFFSET	특수문자 escape·정렬·count 일치
ETL 상세·상품	SHA-256·nullable 처리, 상품 페이지	404·request ID·호출 횟수 AppTest
Promotion History	상태 필터·목록/상세·Audit 분리 조회	목록에서 Audit N+1 회피
Cache 갱신	Promotion 성공 시 history cache 만 무효화	성공 직후 최신 run 이 즉시 표시
반응형 UI	핵심 버튼·상태 계층·간격 정리	Playwright 1440px / 390px 실제 브라우저 확인

Chromium Browser E2E

2 개 시나리오 GitHub Actions browser-e2e	실제 Chromium 버튼·checkbox·페이지 이동	DB 최종 검증 운영 상품·Run·Audit 확인	조회 안전성 History/Audit 전후 DB 불변
---	--	---------------------------------------	---

Promotion E2E — reject batch 와 promotion batch fixture 를 준비하고 Preview → 승인 → 실행 → History/Audit 까지 조작한 뒤 PostgreSQL 최종 상태를 확인합니다.

실제 결함 수정 — 브라우저 E2E 에서 hidden checkbox selector 와 batch 선택 assertion 문제를 발견해 테스트가 실제 UI 구조를 따라가도록 수정했습니다.

읽기 전용 보장 — History 와 Audit 조회 전후 DB 상태가 동일함을 확인하여 조회 기능이 운영 데이터를 변경하지 않도록 검증했습니다.

API Client 방어

CatalogGuardApiClient

HTTP 오류를 도메인 예외로 매핑하고 응답 schema, lowercase SHA-256 Preview Hash 를 검증합니다. Streamlit 은 이 Client 만 사용하므로 UI 에서 SQL 이나 운영 상품 update 를 직접 실행하지 않습니다.

8. CI·Docker Runtime 검증과 성능 판단

테스트 개수보다 실제 서비스를 연결한 회귀 검증과 과거 장애를 CI에서 다시 잡을 수 있는지를 중요하게 보았습니다.

GitHub Actions 자동 검증

단계	검증 범위
DB·Queue	PostgreSQL 18·Redis 7.4 서비스 시작, Alembic upgrade head
회귀 테스트	E2E 제외 pytest, DB 통합 테스트, Celery Worker/FastAPI readiness
비동기 E2E	작업 제출·polling·결과 저장·동일 CSV 재사용·임시 파일 정리
Streamlit	startup smoke /_score/health HTTP 200
AWS Runtime smoke	Dockerfile.aws build → api/services/workers import → UID 10001 → migration → 기본 CMD Uvicorn → /health 200
Browser E2E	Playwright Chromium 2 개 시나리오

실제 장애 → 재발 방지

Docker build 성공만으로 충분하지 않았던 이유

AWS 용 image 는 build 자체는 성공했지만 services/·workers/가 포함되지 않아 실제 FastAPI 실행 시 ModuleNotFoundError 가 발생했습니다. Dockerfile packaging 을 수정한 뒤 GitHub Actions 에 핵심 import, non-root UID 10001, migration, 기본 CMD, /health 검증을 추가하여 같은 누락이 생기면 CI에서 실패하도록 만들었습니다.

PostgreSQL 성능 판단

기존 PostgreSQL 18 benchmark	결과
데이터 규모	inspection_runs 10,000 · inspection_results 100,000
측정 방식	EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), warmup 2 회 + 측정 7 회
주요 조회 중앙값	동일 CSV 0.018 ms · 목록 0.044 ms · count 0.872 ms · 상세 0.016 ms · 결과 0.029 ms
판단	현재 측정 범위에서 병목 근거 없음 → 불필요한 index/migration 추가하지 않음

※ 성능 수치는 기존 저장 benchmark 를 재검토한 결과이며 운영 트래픽 성능을 보증하지 않습니다. ETL·Promotion history 는 향후 전용 PostgreSQL 데이터가 충분히 쌓인 뒤 별도 측정 후보입니다.

9. 검증 결과, 현재 한계와 30 초 소개

현재 구현된 범위와 아직 검증하지 않은 부분을 분리하여 과장 없이 제출할 수 있도록 정리했습니다.

검증 결과

검증 항목	현재 기준	해석
최근 전체 pytest	1042 passed · 88 skipped · 4 deselected	코드 변경 전후 전체 회귀 기준
GitHub Actions	Run 30737512739 · test / browser-e2e success	DB·Redis·Async E2E·Streamlit·AWS runtime·Chromium 검증
AWS Docker Runtime	build/import/UID 10001/migration/Uvicorn /health success	실제 AWS 재배포가 아닌 Ubuntu runner runtime smoke
실제 AWS staging	2026-07 EC2·RDS·TLS·Docker·FastAPI·Streamlit 검증 경험	비용 절감으로 리소스 중지, 최신 변경 재배포는 미 실행

현재 한계

구분	현재 상태
인증·권한	공개 포트폴리오 MVP 로 인증·권한·rate limit 은 미구현
운영 비동기	Redis·Celery 는 로컬·CI E2E 검증, 운영 배포는 미검증
데이터 유입	합성 공급사 프로필 2 종·수동 선택·CSV 중심
ETL 실행	변환·staging 적재는 CLI 중심, 웹에서 ETL 실행 기능은 없음
대용량	streaming·중분 ETL·동시 트래픽 부하 측정 미지원
AWS 최신 재검증	AWS image 는 CI runtime 검증, 최근 코드 기준 EC2·RDS 재배포는 미실행

30 초 소개

면접 답변

CatalogGuard Lite 는 서로 다른 합성 공급사 CSV 를 JSON 프로필로 표준화하고 누락·중복·가격·카테고리·개인정보 문제를 검수하는 데이터 품질 백엔드 프로젝트입니다. FastAPI·PostgreSQL 로 검수 이력과 ETL staging 을 관리하고 Redis·Celery 비동기 검수, Preview Hash 기반 Catalog Promotion 과 Audit 을 구현했습니다. GitHub Actions 에서는 PostgreSQL·Redis·비동기 E2E·Streamlit·Chromium E2E 와 AWS Docker runtime 까지 자동 검증하여 실제 결함을 회귀 테스트로 남겼습니다.

제출 기준

기준일 2026-08-02 · HEAD edd73ed2 · GitHub Actions 30737512739 success · inspection_version 5